PROJET

Worldwide Weather Watcher

Station météo embarquée sur STM32 NUCLEO-L476RG

Documentation technique

Fonctionnement global du système et flux d'information, matériel et câblage, architecture générale du programme

GROUPE 01

LEBBOUZ Riad

KOUIDER ETTOUMI Mohamed El Amine

HAMCHAOUI Razane

BELOUCHRANI Mustapha

Carte : STM32 NUCLEO-L476RG (Cortex-M4, 80 MHz) · Langage : C (HAL STM32 + FatFs)
Capteurs : DHT11, BH1750, GPS GT-U7, RTC interne · Stockage : carte micro-SD
Version du programme : WWW v1.0 · Octobre 2026

Sommaire

1	Objet du document	public, attendus, vocabulaire
2	Présentation générale du système	rôle, grandeurs, vue d'ensemble
3	Matériel utilisé	fiche de chaque composant
4	Câblage et brochage	tableau des broches, schéma, alimentation
5	Fonctionnement global et flux d'information	schéma, déroulement d'une mesure
6	Architecture générale du programme	modules, boucle, états, passage algorithme → C optimisé
7	Données, fichiers et paramètres	format, carte SD, flash
8	Gestion des erreurs	détection, priorités, codes LED
9	Performances	cadences, stockage, énergie, ressources
10	Limites et évolutions	modules complémentaires

Attendus du livrable couverts

Fonctionnement global du système avec flux d'information (schéma, chapitre 5) et architecture générale du programme (chapitre 6). Le matériel et le brochage complet (chapitres 3 et 4) complètent la description.

1. Objet du document

Ce document décrit le fonctionnement interne du prototype de station météo embarquée **Worldwide Weather Watcher**, réalisé pour l'Agence Internationale pour la Vigilance Météorologique (AIVM). La station est destinée à équiper des navires : elle mesure les paramètres de l'air, les horodate, les géolocalise et les enregistre sur une carte micro-SD.

Il s'adresse au **développeur**, au **technicien de maintenance** et au **tuteur** : toute personne qui doit comprendre, câbler, dépanner ou faire évoluer la station. L'utilisation quotidienne est décrite dans la documentation utilisateur, document séparé.

1.1 Vocabulaire

Terme	Signification
Bouton rouge	Bouton bleu B1 soudé sur la NUCLEO (PC13), nommé « rouge » dans le cahier des charges
Bouton vert	Bouton poussoir externe relié à A4 (PC1)
Appui long	Bouton maintenu 5 s (constante `APPUI_LONG_MS`)
Acquisition	Lecture de tous les capteurs pour produire une ligne de mesures
Super-boucle	Boucle infinie de `main()` qui appelle les tâches à tour de rôle, sans système d'exploitation
Non bloquant	Une fonction rend la main tout de suite au lieu d'attendre (pas de `HAL_Delay` pendant une mesure)
Trame NMEA	Ligne de texte envoyée par le GPS (`$GPGGA`, `$GPRMC`…)
Fix	Position calculée par le GPS (au moins 4 satellites)
NA	« Non disponible » : valeur absente dans une ligne de mesures

2. Présentation générale du système

La station est construite autour d'une carte **STM32 NUCLEO-L476RG** (microcontrôleur ARM Cortex-M4 cadencé à 80 MHz). À intervalle régulier, elle relève :

Grandeur	Capteur	Liaison	Unité enregistrée
Température de l'air	DHT11	1 fil (A3)	°C, entier
Hygrométrie	DHT11	1 fil (A3)	% HR, entier
Luminosité	BH1750	I2C1 (D14/D15)	lux, entier
Position	GPS GT-U7	USART1 (D2/D8)	NMEA DDMM.mmmmm N/S, DDDMM.mmmmm E/W
Date et heure	RTC interne (quartz 32,768 kHz)	–	JJ/MM/AAAA HH:MM:SS

Chaque relevé produit **une seule ligne** de texte horodatée, ajoutée au fichier du jour sur la carte micro-SD. L'utilisateur agit sur la station avec **deux boutons**, lit son état sur une **LED RGB** et la paramètre depuis une **console série**.

2.1 Vue d'ensemble

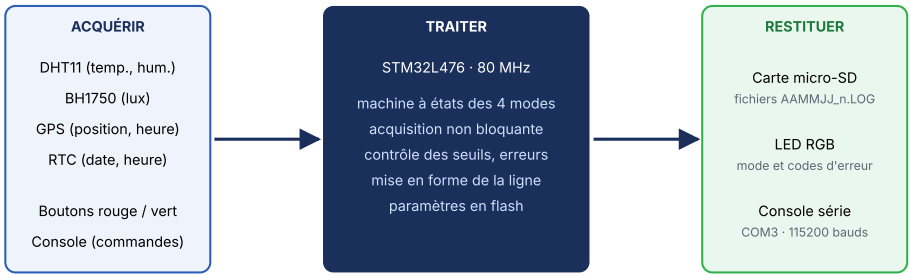


Figure 1 : chaîne acquérir → traiter → restituer

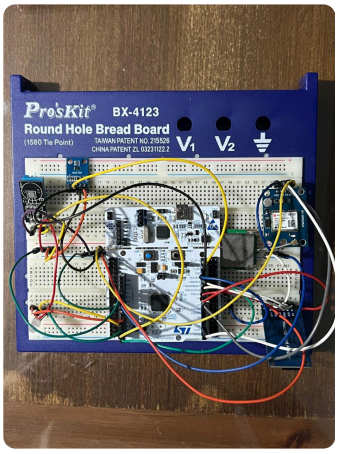
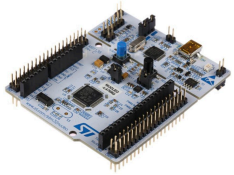


Figure 2 : la maquette assemblée

> **Périmètre du prototype**
> Le capteur de pression atmosphérique cité par le sujet n'était pas disponible : la grandeur a été retirée du programme. Les modules complémentaires (température de l'eau, courant marin, vent, particules fines) ne sont pas encore intégrés, mais l'architecture prévoit leur ajout (chapitre 10).

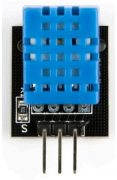
3. Matériel utilisé

Chaque composant a été choisi pour sa simplicité de câblage avec la NUCLEO et pour sa disponibilité dans le kit. Le brochage détaillé est donné au chapitre 4.



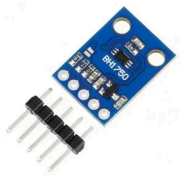
Carte STM32 NUCLEO-L476RG

Rôle	Unité de traitement : exécute le programme, pilote tous les périphériques
Caractéristiques	Cortex-M4 à 80 MHz, 1 Mo de flash, 128 Ko de RAM, RTC intégrée, sonde ST-Link
Interfaces	USB ST-Link : alimentation, console série (USART2) et téléversement (SWD) par un seul câble
Pourquoi	Imposée par le sujet ; famille basse consommation (L4), RTC avec quartz 32,768 kHz, bouton B1 intégré



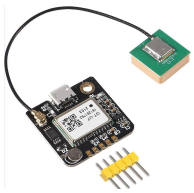
Capteur DHT11 (température, humidité)

Rôle	Mesure la température (0 à 50 °C, ±2 °C) et l'humidité (20 à 90 % HR, ±5 %)
Liaison	Protocole 1 fil propriétaire, trame de 40 bits avec somme de contrôle
Broches	S (DATA) → A3 · + → 3V3 · – → GND
Pourquoi	Deux grandeurs avec une seule broche ; décodage simple avec le compteur de cycles DWT (µs)



Capteur de luminosité BH1750

Rôle	Mesure l'éclairement de 1 à 65 535 lux, résolution 1 lx
Liaison	Bus I2C1 à 100 kHz, adresse 0x23 (broche ADDR à GND)
Broches	SCL → D15 · SDA → D14 · VCC → 3V3 · GND → GND
Pourquoi	Donne directement des lux sans calibration ; le bus I2C pourra accueillir les futurs modules



Module GPS GT-U7 (avec antenne)

Rôle	Donne la position (trame GGA) et l'heure UTC (trame RMC) qui sert à régler la RTC
Liaison	UART 9600 bauds, 8N1, trames NMEA, une salve par seconde
Broches	TX → D2 · RX → D8 · VCC → 5V · GND → GND
Pourquoi	Trames standard ; l'heure GPS remet la RTC à l'heure après une coupure (pas de pile sur la carte)

Lecteur de carte micro-SD



Rôle	Stockage des mesures sur une carte micro-SD FAT32, lisible sur tout PC
Liaison	SPI1 à 312,5 kbit/s (80 MHz / 256), système de fichiers FatFs
Broches	CS → D10 · MOSI → D11 · MISO → D12 · SCK → D13 · VCC → 5V · GND → GND
Pourquoi	Fichiers texte directement exploitables ; SPI1 car PA5/PA6/PA7 ne sont reliées qu'à SPI1 ; horloge lente pour la fiabilité avec des fils Dupont

LED RGB 4 pattes, anode commune



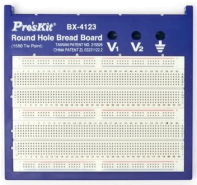
Rôle	Affiche le mode (couleur fixe) et les codes d'erreur (clignotement à 1 Hz)
Liaison	3 sorties GPIO pilotées en PWM logiciel (10 niveaux) depuis le SysTick
Broches	R → 220 Ω → A0 · V → 100 Ω → A1 · B → 100 Ω → A2 · patte longue (anode) → 3V3
Pourquoi	Une seule LED pour 4 modes et 6 codes ; le PWM logiciel donne jaune, orange et blanc sans timer matériel

Bouton poussoir 6×6 mm (bouton vert)



Rôle	Bouton « vert » du cahier des charges : passage en mode économique
Liaison	Entrée GPIO avec pull-up interne : relâché = 1, appuyé = 0 ; anti-rebond logiciel 30 ms
Broches	Deux pattes en diagonale : l'une → A4, l'autre → GND (aucune résistance)
Pourquoi	Le second bouton (« rouge ») est B1, déjà soudé sur la carte : un seul composant à ajouter

Plaque d'essai Pro'sKit BX-4123



Rôle	Support de la maquette : porte la NUCLEO, la LED, le bouton et les modules, reliés par des fils Dupont
Organisation	Rangées de 5 trous reliés, rainure centrale, rails + et – sur toute la longueur
Rails	Un rail 3V3 (capteurs, LED) et un rail 5V (GPS, lecteur SD), masses communes
Pourquoi	Montage sans soudure, modifiable pendant la mise au point

4. Câblage et brochage

Tous les modules sont reliés aux connecteurs Arduino de la NUCLEO par des fils Dupont. La carte est alimentée par le câble USB du ST-Link, qui sert aussi à la console série et au téléversement. Le brochage ci-dessous correspond à la configuration CubeMX (`StationMeteo.ioc`) du programme.

4.1 Tableau de brochage

Module	Broche du module	NUCLEO	STM32	Périphérique
DHT11	DATA (S)	**A3**	PB0	GPIO, 1 fil
	VCC / GND	3V3 / GND	–	–
BH1750	SCL / SDA	**D15 / D14**	PB8 / PB9	I2C1, 100 kHz, adr. 0x23
	VCC / GND	3V3 / GND	–	–
GPS GT-U7	TX (sortie du GPS)	**D2**	PA10 (RX)	USART1, 9600 bauds
	RX (entrée du GPS)	**D8**	PA9 (TX)	USART1
	VCC / GND	5V / GND	–	–
Lecteur SD	CS	**D10**	PB6	GPIO (sélection)
	MOSI / MISO / SCK	**D11 / D12 / D13**	PA7 / PA6 / PA5	SPI1, 312,5 kbit/s
	VCC / GND	5V / GND	–	–
LED RGB	R / V / B (via résistance)	**A0 / A1 / A2**	PA0 / PA1 / PA4	GPIO, PWM logiciel
	Anode commune	3V3	–	–
Bouton vert	2 pattes en diagonale	**A4** et GND	PC1	GPIO, pull-up interne
Bouton rouge	B1, soudé sur la carte	–	PC13	EXTI13, pull-up
Console	USB du ST-Link	port COM3 du PC	PA2 / PA3	USART2, 115200 bauds

Points de vigilance

- **Croiser TX et RX** : la sortie TX du GPS va sur l'entrée RX de la carte (D2), et inversement (D8).
- **Ne pas utiliser D0/D1** pour le GPS : sur la NUCLEO, ces broches sont réservées à la liaison USB du ST-Link.
- Le GPS et le lecteur SD sont alimentés en **5V** : vérifier que le rail 5V de la plaque est bien relié (cause d'une panne SD rencontrée pendant les essais).
- PA5 (D13) est aussi la LED verte LD2 de la carte : elle clignote avec l'horloge SPI, c'est normal.
- LED à **anode commune** : une couleur s'allume quand sa broche est à 0. L'inversion est gérée par `LED_ANODE_COMMUNE` dans `led.h` .

4.2 Schéma de câblage

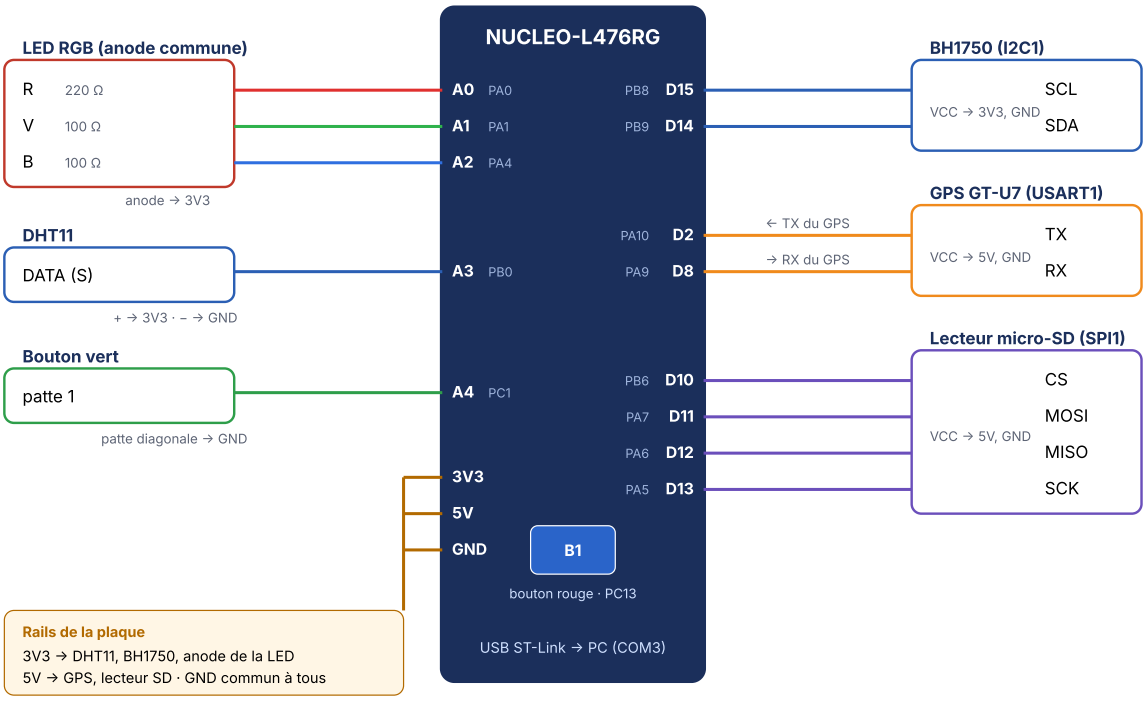


Figure 3 : schéma de câblage (broche Arduino en blanc, broche STM32 en bleu clair)

4.3 Alimentation

Rail	Modules alimentés	Remarque
USB 5 V	Carte NUCLEO	PC ou batterie externe, par le connecteur du ST-Link
3V3 (régulateur de la carte)	DHT11, BH1750, anode de la LED RGB	Capteurs compatibles 3,3 V : niveaux logiques directs avec le STM32
5V	GPS GT-U7, lecteur micro-SD	Les deux modules ont leur propre régulateur 3,3 V et des niveaux logiques compatibles
GND	Tous	Masse commune obligatoire pour les liaisons série, I2C et SPI

Calcul des résistances de la LED : rouge, chute ≈ 2 V, (3,3 − 2) / 220 ≈ 6 mA ; vert et bleu, chute ≈ 2,8 à 3 V, 100 Ω donnent 3 à 5 mA. On reste très en dessous des 20 mA admis par une broche du STM32.

5. Fonctionnement global et flux d'information

Le schéma ci-dessous montre le chemin des informations, des capteurs jusqu'aux sorties de la station. Les flèches **bleues pleines** sont les données de mesure ; les flèches **orange en pointillés** sont les informations de commande (paramètres, mode, heure).

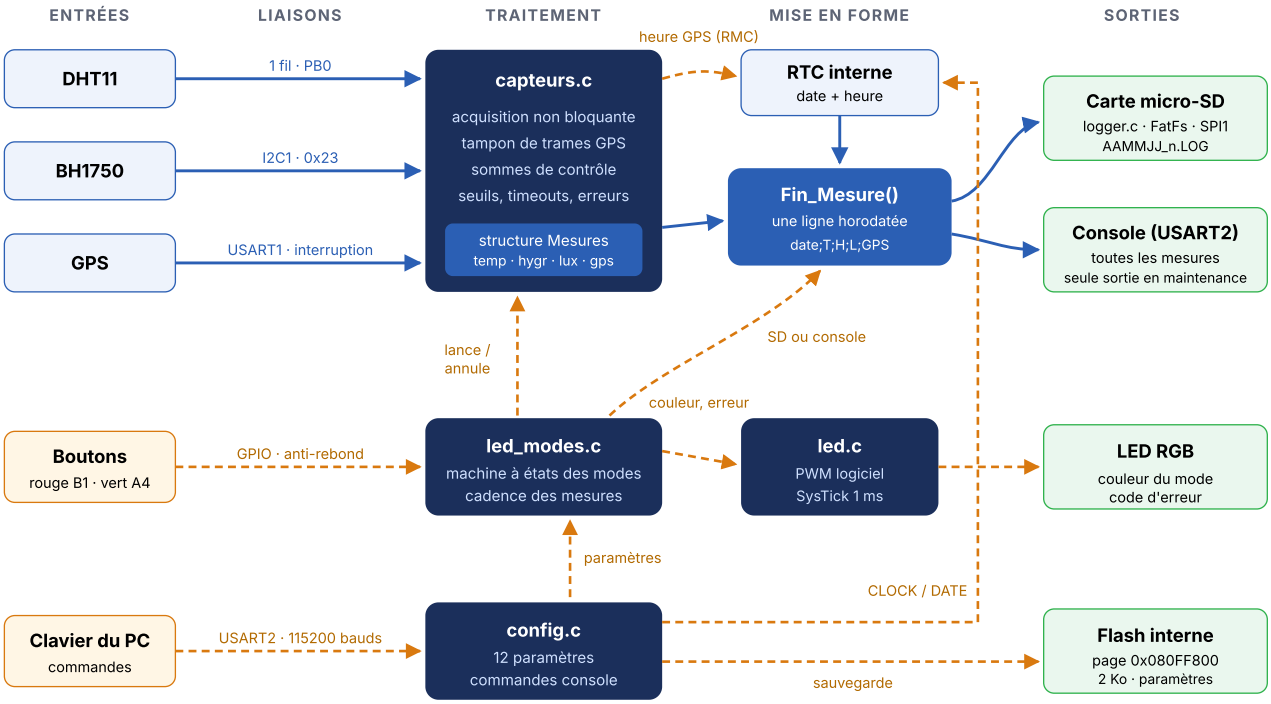


Figure 4 : flux d'information de la station (données de mesure en bleu, commandes en orange)

5.1 Déroulement d'une mesure

1. **Déclenchement.** `led_modes.c` lance une mesure dès l'entrée dans un mode de mesure, puis à chaque période (`LOG_INTERVAL` , doublé en économique, 5 s en maintenance). L'heure de début est lue dans la RTC.
2. **Acquisition.** À chaque tour de boucle, `Capteurs_Tache()` tente une lecture de chaque capteur pas encore lu : DHT11 toutes les 2 s, BH1750 toutes les 200 ms, GPS dès qu'une trame GGA valide (somme de contrôle juste, fix présent) a été reçue sous interruption.
3. **Fin.** L'acquisition se termine quand tout est lu ou quand `TIMEOUT` est écoulé. Les compteurs d'échecs et les erreurs sont mis à jour.
4. **Mise en forme.** `Fin_Mesure()` construit une ligne unique : horodatage + valeurs, `NA` pour une valeur absente.
5. **Sortie.** La ligne est ajoutée au fichier du jour sur la carte SD et affichée sur la console, ou seulement affichée en mode maintenance. La LED est mise à jour selon l'erreur la plus grave.

5.2 Réglage de l'heure par le GPS

La carte n'a pas de pile de sauvegarde : l'heure est perdue quand l'USB est débranché (elle est conservée au RESET). Dès qu'une trame RMC contient une heure valide (un seul satellite suffit, pas besoin de fix), le programme règle la RTC en heure locale (UTC+1). L'opérateur peut aussi la régler à la main avec `CLOCK=` et `DATE=` en mode configuration.

6. Architecture générale du programme

Le programme est écrit en C sur la bibliothèque HAL de ST, sans système d'exploitation. Il est découpé en **modules** : un fichier `.c` et son en-tête `.h` par responsabilité. Les modules du haut ne connaissent les composants qu'à travers les modules du dessous, ce qui permet de changer un capteur sans toucher à la logique des modes.

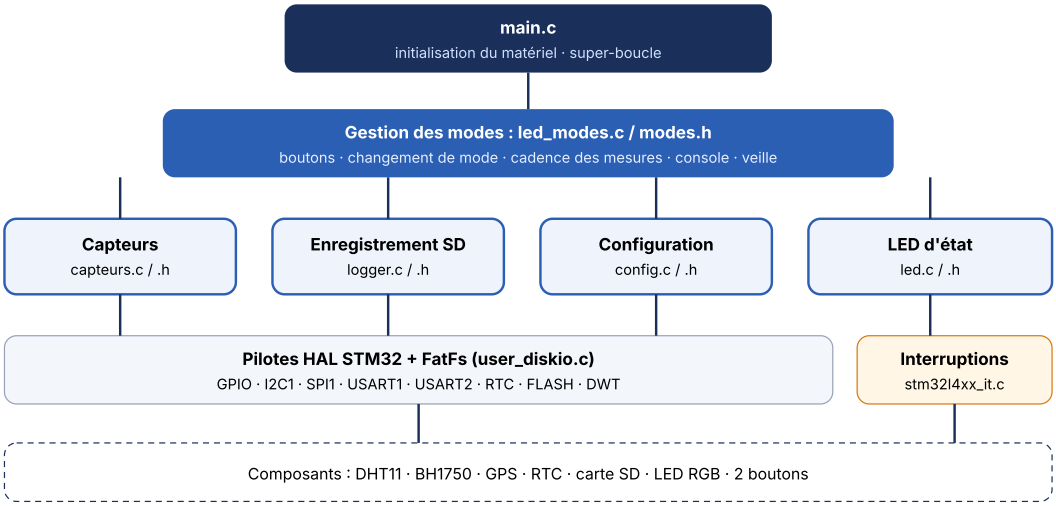


Figure 5 : découpage du programme en couches et dépendances

6.1 Rôle des modules

Module	Responsabilité	Fonctions principales
`main.c`	Horloges (80 MHz), initialisation des périphériques générée par CubeMX, boucle infinie	`main()`, `SystemClock_Config()`, `MX_xxx_Init()`
`led_modes.c`	Machine à états des 4 modes, boutons (anti-rebond, appui long), période des mesures, console, mise en veille	`LedModes_Init()`, `LedModes_Loop()`, `Changer()`, `Mesure()`, `Fin_Mesure()`
`capteurs.c`	Pilotage des capteurs, réception GPS, contrôle des seuils, compteurs de timeouts, formatage des valeurs	`Capteurs_Demarrer()`, `Capteurs_Tache()`, `Capteurs_GPS_IRQ()`, `Capteurs_Formater()`
`logger.c`	Montage de la carte SD, écriture des lignes, rotation des fichiers pleins	`Logger_Monter()`, `Logger_Ecrire()`, `Logger_Demonter()`
`config.c`	Table des paramètres et bornes, sauvegarde en flash, commandes de la console, réglage de la RTC	`Config_Charger()`, `Config_Commande()`
`led.c`	Couleur du mode et clignotement des codes d'erreur, PWM logiciel	`LED_Couleur()`, `LED_Erreur()`, `LED_Tick()`
`stm32l4xx_it.c`	Routines d'interruption, qui appellent les modules	`SysTick_Handler`, `USART1_IRQHandler`

`user_diskio.c`	Pilote bas niveau de la carte SD en mode SPI pour FatFs	lecture/écriture de secteurs

6.2 Organisation des fichiers

```
StationMeteo/
├── StationMeteo.ioc          // configuration CubeMX : broches, horloges, périphériques
├── CMakeLists.txt            // projet CMake (compilation arm-none-eabi-gcc)
├── Core/Inc/                 // en-têtes : capteurs.h, config.h, led.h, logger.h, modes.h
…
├── Core/Src/
│   ├── main.c                // initialisation + super-boucle
│   ├── led_modes.c           // modes, boutons, cadence, console
│   ├── capteurs.c            // DHT11, BH1750, GPS
│   ├── logger.c              // carte SD, fichiers LOG
│   ├── config.c              // paramètres, flash, commandes
│   ├── led.c                 // LED RGB, codes d'erreur
│   └── stm32l4xx_it.c        // interruptions
└── FATFS/Target/user_diskio.c  // pilote SD en SPI
```

6.3 Principe d'exécution : la super-boucle

Après l'initialisation, `main()` appelle `LedModes_Loop()` sans fin. Le temps est mesuré avec `HAL_GetTick()` (millisecondes depuis le démarrage) : chaque action compare l'heure actuelle à l'heure de sa dernière exécution, ce qui évite toute attente bloquante. Les boutons et la LED restent donc réactifs pendant une mesure.

```
PROCÉDURE main()
    initialiser horloges (80 MHz), GPIO, I2C1, SPI1, USART1, USART2, RTC
    LedModes_Init()      // charge les paramètres, lit B1 : CONFIGURATION ou STANDARD
    RÉPÉTER SANS FIN LedModes_Loop()

PROCÉDURE LedModes_Loop()
    lire les boutons (anti-rebond 30 ms, appui long 5 s)
    régler l'heure avec le GPS si elle n'est pas réglée
    SELON mode : changer de mode selon les appuis longs   // diagramme d'états 6.5
    SI mode = CONFIGURATION ALORS traiter la console ; RETOURNER
    SI une mesure est en cours ALORS
        SI Capteurs_Tache() ALORS Fin_Mesure()        // écrire la ligne
    SINON SI la période est écoulée ALORS Mesure()       // lancer une mesure
    SI aucune mesure en cours ALORS __WFI()             // veille jusqu'à la prochaine
interruption
FIN
```

6.4 Interruptions

Interruption	Fréquence	Traitement
SysTick	1 ms	Base de temps HAL et `LED_Tick()` : PWM logiciel à 100 Hz et clignotement des codes d'erreur à 1 Hz
USART1 (GPS)	à chaque caractère (9600 bauds)	`Capteurs_GPS_IRQ()` : remplit le tampon de trame, copie la dernière trame complète et la dernière trame RMC ; les erreurs de réception sont effacées sans arrêter la réception

EXTI13 (B1)	sur appui	Réveil du processeur ; l'état du bouton est ensuite lu par scrutation dans la boucle

6.5 Machine à états des modes

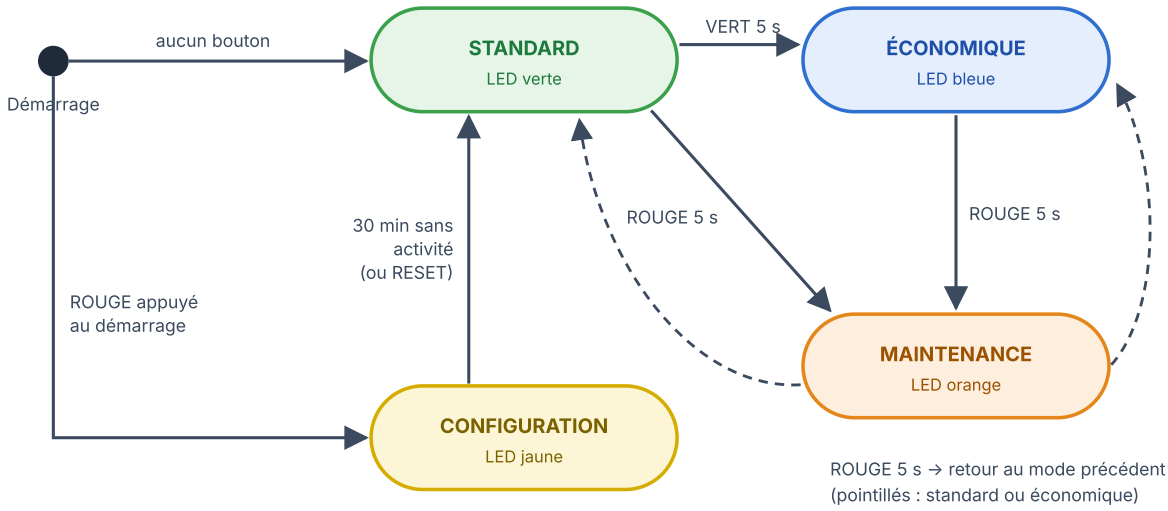


Figure 6 : diagramme d'états. Bouton « rouge » = B1 (PC13) ; bouton « vert » = bouton miniature (A4). En pointillés : retour au mode précédent.

Mode	LED	Acquisition	Enregistrement	Particularités
Standard	🟢 verte	toutes les `LOG_INTERVAL` min, GPS à chaque mesure	carte SD	mode par défaut
Économique	🔵 bleue	toutes les 2 × `LOG_INTERVAL`, GPS une mesure sur deux	carte SD	réduit la consommation
Maintenance	🟠 orange	toutes les 5 s	console seulement	SD démontée, retrait possible ; mémorise le mode précédent
Configuration	🟡 jaune	désactivée	–	console de commandes ; retour en standard après 30 min

À chaque changement de mode, `Changer()` abandonne la mesure en cours (`Capteurs_Annuler()`), démonte ou remonte la carte SD, change la couleur de la LED et déclenche une mesure immédiate.

6.6 Acquisition non bloquante

`capteurs.c` ne contient aucune attente longue : chaque appel de `Capteurs_Tache()` fait au plus une tentative par capteur, puis rend la main.

Capteur	Cadence des tentatives	Lecture
DHT11	toutes les 2 s (contrainte du capteur), 1,5 s après le démarrage	Impulsion de démarrage, puis 40 bits mesurés au compteur DWT (µs), interruptions masquées environ 5 ms ; somme de contrôle vérifiée, trame vide rejetée

BH1750	toutes les 200 ms	Commande de mesure haute résolution sur I2C1, lecture de 2 octets, conversion en lux (÷ 1,2)
GPS	à chaque trame reçue	Trame GGA : somme XOR vérifiée, champ « fix » non nul, extraction latitude/longitude. Non lue une mesure sur deux en économique

L'acquisition s'arrête quand toutes les grandeurs demandées sont lues, ou à `TIMEOUT`. Un capteur qui échoue deux acquisitions de suite lève une erreur (chapitre 8) ; sa valeur est notée `NA`.

6.7 Enregistrement sur la carte SD

- `Logger_Ecrire()` ouvre le fichier du jour `AAMMJJ_0.LOG`, ajoute la ligne puis **ferme le fichier** : une coupure de courant ne fait perdre au plus que la ligne en cours.
- Si la ligne ferait dépasser `FILE_MAX_SIZE`, le fichier est renommé dans la première révision libre (`_1` à `_9`) et un nouveau `_0` est créé.
- Après une erreur, la carte est démontée et le programme tente de la remonter à l'écriture suivante. Au premier démarrage, le montage est réessayé plusieurs fois.
- En maintenance, `Logger_Demonter()` libère la carte ; elle est remontée à la sortie du mode.

6.8 Console et paramètres

La console est lue sans bloquer dans la boucle (registre de réception d'USART2). Chaque ligne tapée est mise en majuscules puis passée à `Config_Commande()`, qui reconnaît `HELP`, `PARAMS`, `VERSION`, `CLOCK`, `DATE`, `DAY`, `RESET` et `NOM=valeur` pour chaque paramètre de la table. Chaque valeur est contrôlée par rapport à ses bornes avant d'être écrite en flash.

6.9 Du Livrable 02 (algorithmique) au programme C optimisé

Le Livrable 02 décrivait le programme en **algorithmique** : structure générale, modes, acquisition, carte SD, console et LED. Pour la maquette, tout ce programme a été **écrit en C** (bibliothèque HAL de ST et FatFs), testé sur la carte, puis **optimisé**. L'architecture du Livrable 02 a été conservée : mêmes modules, mêmes noms de fonctions, même machine à états. Ce qui a changé, ce sont les détails d'écriture qui rendent le programme plus sûr, plus rapide et moins gourmand.

Cette section reprend chaque partie du programme : l'algorithme du Livrable 02, le code C final (extrait tel quel des fichiers du projet), puis ce qui a changé et pourquoi.

Correspondance algorithme → C

Algorithme (Livrable 02)	Fonction C (fichier)	Changement principal
`StationMeteo_WWW`	`main()` (main.c)	Choix test / station à la compilation (`#if TEST_xxx`)
`Modes_Init`, `Modes_Boucle`	`LedModes_Init()`, `LedModes_Loop()` (led_modes.c)	Veille `__WFI()` et heure GPS ajoutées
`Appui_Long`, `Changer_Mode`, `Periode`	`AppuiLong()`, `Changer()`, `Periode()`	Passage par pointeur, entiers non signés 32 bits
`Capteurs_Tache`, `Fin_Acquisition`	mêmes noms (capteurs.c)	Délai de démarrage des capteurs, pression retirée
`DHT11_Lire`	`DHT11_Lire()`	Temps en µs par le compteur DWT, trame vide rejetée
`BH1750_Lire`	`BH1750_Lire()`	Calcul en entiers, délai I2C de 10 ms
`Capteurs_GPS_IRQ`, `GPS_Lire_Position`	mêmes noms + `GPS_Checksum_OK()`, `GPS_Regler_RTC()`	USART1, somme de contrôle, heure automatique
`Logger_Monter`, `Logger_Ecrire`, `Archiver`	mêmes noms (logger.c)	Montage immédiat, contrôle des octets écrits
`Config_Commande`, `Config_Sauver`	mêmes noms (config.c)	Table de paramètres par pointeurs, format flash v2
`LED_Tick`	`LED_Tick()`, `Ecrire()` (led.c)	Une seule écriture de registre (BSRR)

1. La boucle principale

Livrable 02

```
PROCEDURE Modes_Boucle()
    long_rouge ← Appui_Long(rouge, maintenant)
    long_vert  ← Appui_Long(vert, maintenant)
    SELON mode … (transitions)
    SI Capteurs_EnCours() ALORS
        SI Capteurs_Tache() ALORS Fin_Mesure()
    SINON SI mesure_immediate OU maintenant - t_mesure ≥ Periode() ALORS
        Mesure()
FIN
```

Programme C : led_modes.c

```c
void LedModes_Loop(void)
{
  uint32_t now = HAL_GetTick();
  uint8_t long_rouge = AppuiLong(&rouge, now); /* B1 */
  uint8_t long_vert = AppuiLong(&vert, now);  /* bouton miniature */

  Capteurs_GPS_Horloge(); /* heure automatique depuis le GPS après une coupure de courant */

  switch (mode)
  {
    case MODE_CONFIGURATION:
      Console(now);
      if (rouge.enfonce || vert.enfonce) t_activite = now;
      if (now - t_activite >= CONFIG_TIMEOUT_MS)
      {
        printf("\r\n30 min sans activite\r\n");
        Changer(MODE_STANDARD);
      }
      return; /* pas d'acquisition en configuration */

    case MODE_STANDARD:
      if (long_rouge)     Changer(MODE_MAINTENANCE);
      else if (long_vert) Changer(MODE_ECONOMIQUE);
      break;

    case MODE_ECONOMIQUE:
      if (long_rouge)   Changer(MODE_MAINTENANCE);
      break;

    case MODE_MAINTENANCE:
      if (long_rouge)   Changer(mode_precedent);
      break;
  }

  if (Capteurs_EnCours())
  {
    if (Capteurs_Tache()) Fin_Mesure();
  }
  else if (mesure_immediate || now - t_mesure >= Periode())
  {
    mesure_immediate = 0;
    t_mesure = now;
    Mesure();
  }

  /* Économie d'énergie : entre deux mesures, le processeur dort jusqu'à la prochaine
   * interruption (SysTick toutes les 1 ms : LED et boutons ; réception GPS). Pas de
   * veille pendant une mesure (capteurs lus en scrutation) ni en configuration
   * (console lue sans interruption, on a quitté la fonction plus haut). */
  if (!Capteurs_EnCours()) __WFI();
}
```

Ce qui a changé et pourquoi

- **Mise en veille** `__WFI()` à la fin de chaque tour quand aucune mesure n'est en cours. Dans le Livrable 02, la boucle tournait sans arrêt, des millions de fois par seconde, pour rien. Maintenant le processeur dort et ne se réveille qu'à une interruption (SysTick 1 ms, caractère GPS, bouton). C'était l'optimisation annoncée dans la conclusion du Livrable 02.
- `Capteurs_GPS_Horloge()` est appelée à chaque tour : l'heure est réglée par le GPS dès qu'il capte un satellite, même entre deux mesures. La carte n'ayant pas de pile, c'est ce qui évite les fichiers datés du 01/01/2000.
- `HAL_DBGMCU_EnableDBGSleepMode()` (dans `LedModes_Init()`) garde le ST-Link connecté pendant la veille, sinon on ne pourrait plus reprogrammer la carte facilement.

2. Les boutons et l'appui long

Livrable 02

```
FONCTION Appui_Long(b : Bouton, maintenant : entier) : booléen
    lecture ← (Lire_Broche(b.port, b.broche) = 0)
    SI lecture ≠ b.brut ALORS b.brut ← lecture ; b.t_change ← maintenant
    SI b.enfonce ≠ b.brut ET maintenant - b.t_change ≥ ANTI_REBOND_MS ALORS …
    SI b.enfonce ET NON b.long_fait ET maintenant - b.t_appui ≥ APPUI_LONG_MS ALORS
        b.long_fait ← VRAI ; RETOURNER VRAI
```

Programme C : led_modes.c

```c
/* Renvoie 1 une seule fois, au moment où le bouton atteint 5 s d'appui */
static uint8_t AppuiLong(Bouton *b, uint32_t now)
{
  uint8_t brut = Lire(b);

  if (brut != b->brut)
  {
    b->brut = brut;
    b->t_change = now;
  }
  if (b->enfonce != b->brut && now - b->t_change >= ANTI_REBOND_MS)
  {
    b->enfonce = b->brut;
    printf("[BOUTON] %s %s\r\n", (b->pin == BTN1_Pin) ? "rouge (B1)" : "vert (miniature A4)", b-
>enfonce ? "appuye" : "relache");
    if (b->enfonce)
    {
      b->t_appui = now;
      b->long_fait = 0;
    }
  }
  if (b->enfonce && !b->long_fait && now - b->t_appui >= APPUI_LONG_MS)
  {
    b->long_fait = 1;
    return 1;
  }
  return 0;
}
```

Ce qui a changé et pourquoi

- Le bouton est passé **par pointeur** (`Bouton *b`) : la fonction modifie directement la structure sans la recopier.
- Les temps sont des **`uint32_t`** et on calcule toujours une **différence** `now - t_change` : le calcul reste juste même quand `HAL_GetTick()` repasse à zéro (après 49 jours de fonctionnement).
- Le message `[BOUTON] … appuye / relache` a été ajouté pour vérifier le câblage depuis la console.

3. Cadence des mesures

Livrable 02

```
FONCTION Periode() : entier
    p ← cfg.log_interval × 60 000
    SI mode = MODE_ECONOMIQUE ALORS p ← p × 2
    SI mode = MODE_MAINTENANCE ALORS p ← MAINTENANCE_PERIODE_MS
    RETOURNER p
```

Programme C : led_modes.c

```c
/* Intervalle entre 2 mesures selon le mode */
static uint32_t Periode(void)
{
  uint32_t p = (uint32_t)cfg.log_interval * (LOG_INTERVAL_EN_SECONDES ? 1000UL : 60000UL);

  if (mode == MODE_ECONOMIQUE) p *= 2;              /* LOG_INTERVAL x 2 */
  if (mode == MODE_MAINTENANCE) p = MAINTENANCE_PERIODE_MS;
  return p;
}
```

Ce qui a changé et pourquoi

- Le calcul est fait en **entiers 32 bits non signés** (`60000UL`) : 1440 min × 60 000 = 86 400 000 ms tient sans débordement, ce qui ne serait pas le cas sur 16 bits.
- La constante `LOG_INTERVAL_EN_SECONDES` permet, pour une démonstration, de compter `LOG_INTERVAL` en secondes. Comme c'est un `#define` , le compilateur supprime la branche inutile : aucun coût à l'exécution.

4. Acquisition non bloquante des capteurs

Livrable 02

```
FONCTION Capteurs_Tache() : booléen
    SI NON dht_fini ET maintenant - t_dht ≥ 2000 ALORS … DHT11_Lire(t, h)
    SI NON lum_fini ET maintenant - t_lum ≥ 200 ALORS … BH1750_Lire(lux)
    mesures.pres_ok ← FAUX              // pression : pas de capteur
    SI NON gps_fini ET GPS_Lire_Position() ALORS …
    SI tout est lu OU TIMEOUT écoulé ALORS Fin_Acquisition() ; RETOURNER VRAI
```

Programme C : capteurs.c

```c
uint8_t Capteurs_Tache(void)
{
  uint32_t now = HAL_GetTick();
  int16_t t, h;
  uint16_t lux;

  if (!en_cours) return 1;
  if (now - t_init < DEMARRAGE_MS) return 0; /* capteurs pas encore prêts juste après le reset */
```

```
  /* --- DHT11 : une tentative toutes les 2 s jusqu'à réussite --- */
  if (!dht_fini && now - t_dht >= DHT11_PERIODE_MS)
  {
    t_dht = now;
    if (DHT11_Lire(&t, &h))
    {
      mesures.temp = t;
      mesures.temp_ok = cfg.temp_air ? 1 : 0;
      /* l'hygrométrie n'est enregistrée que si la température est dans HYGR_MINT..HYGR_MAXT */
      mesures.hygr = h;
      mesures.hygr_ok = (cfg.hygr && t >= cfg.hygr_mint && t <= cfg.hygr_maxt) ? 1 : 0;
      dht_fini = 1;
    }
  }

  /* --- BH1750 --- */
  if (!lum_fini && now - t_lum >= BH1750_PERIODE_MS)
  {
    t_lum = now;
    if (BH1750_Lire(&lux))
    {
      mesures.lux = lux;
      mesures.lum_ok = 1;
      lum_fini = 1;
    }
  }

  /* --- GPS --- */
  if (!gps_fini && GPS_Lire_Position())
  {
    mesures.gps_ok = 1;
    gps_fini = 1;
  }

  /* --- Fin : tout est lu, ou TIMEOUT écoulé --- */
  if ((dht_fini && lum_fini && gps_fini) || now - t_debut >= (uint32_t)cfg.timeout * 1000UL)
  {
    Fin_Acquisition();
    return 1;
  }
  return 0;
}
```

Ce qui a changé et pourquoi

- **Délai de démarrage** `DEMARRAGE_MS` (1,5 s) : juste après un RESET, le DHT11 et le BH1750 ne sont pas prêts. Sans ce délai, la première mesure comptait un faux échec.
- **Pression retirée** : le capteur n'existe pas dans le kit. Le champ, les 3 paramètres `PRESSURE*` et la colonne de la ligne ont été supprimés, ce qui donne une ligne plus courte et moins de code.
- Dans `Capteurs_Demarrer()`, `t_dht = now - DHT11_PERIODE_MS` fait la première tentative **tout de suite**, au lieu d'attendre 2 s.

5. Lecture du DHT11

Livrable 02

```
Ecrire_Broche(PB0, 0) ; Attendre(18 ms) ; Ecrire_Broche(PB0, 1) ; Attendre_us(30)
Desactiver_Interruptions()
… lecture des 40 bits (Attendre_Niveau, Attendre_us(40)) …
Activer_Interruptions()
SI octets[4] = somme des 4 octets MOD 256 ALORS … RETOURNER VRAI
```

Programme C : capteurs.c

```c
/* Attend tant que PB0 vaut 'etat' ; 0 si ça dure plus de timeout_us */
static uint8_t DHT11_Attendre(GPIO_PinState etat, uint32_t timeout_us)
{
  uint32_t start = DWT->CYCCNT;
  uint32_t max = timeout_us * (SystemCoreClock / 1000000);
  while (HAL_GPIO_ReadPin(DHT11_GPIO_Port, DHT11_Pin) == etat)
  {
    if ((DWT->CYCCNT - start) > max) return 0;
  }
  return 1;
}

static uint8_t DHT11_Lire(int16_t *temperature, int16_t *humidite)
{
  uint8_t o[5], ok = 1;

  /* 1. Signal de départ : ligne à 0 pendant 18 ms */
  DHT11_Sortie();
  HAL_GPIO_WritePin(DHT11_GPIO_Port, DHT11_Pin, GPIO_PIN_RESET);
  HAL_Delay(18);
  HAL_GPIO_WritePin(DHT11_GPIO_Port, DHT11_Pin, GPIO_PIN_SET);
  delay_us(30);
  DHT11_Entree();

  /* 2. Réponse du capteur, puis 3. les 40 bits, sans interruptions */
  __disable_irq();
  if (!DHT11_Attendre(GPIO_PIN_SET, 1000) || !DHT11_Attendre(GPIO_PIN_RESET, 1000) ||
      !DHT11_Attendre(GPIO_PIN_SET, 1000))
  {
    __enable_irq();
    return 0;
  }
  for (int i = 0; i < 5; i++) o[i] = DHT11_LireOctet(&ok);
  __enable_irq();

  /* 4. Somme de contrôle */
  if (!ok || o[4] != ((o[0] + o[1] + o[2] + o[3]) & 0xFF)) return 0;
  if (o[0] == 0 && o[2] == 0) return 0; /* trame vide (capteur pas prêt) : la somme 0 passerait le
contrôle */
  *humidite = o[0];
  *temperature = o[2];
  return 1;
}
```

Ce qui a changé et pourquoi

- Les attentes en microsecondes utilisent le **compteur de cycles DWT** du Cortex-M4 (80 cycles = 1 µs). `HAL_Delay` ne compte qu'en millisecondes, trop lent pour un bit du DHT11 (26 à 70 µs).
- Chaque attente a un **délai maximal** (1000 µs) : si le capteur est débranché, la fonction rend la main au lieu de bloquer la station.
- Une **trame vide** (tout à 0) est rejetée : sa somme de contrôle vaut 0, elle passait donc le contrôle du Livrable 02 et donnait 0 °C / 0 %.
- Les interruptions ne sont coupées que pendant les 40 bits (environ 5 ms), pas pendant le signal de départ de 18 ms.

6. Lecture du BH1750

Livrable 02

```
SI I2C_Recevoir(0x23, donnees, 2, cfg.timeout) = OK ALORS
    lux ← (donnees[0] × 256 + donnees[1]) / 1,2 ; RETOURNER VRAI
SINON BH1750_Init() ; RETOURNER FAUX
```

Programme C : capteurs.c

```c
static uint8_t BH1750_Lire(uint16_t *lux)
{
  uint8_t rx[2] = {0};
  if (HAL_I2C_Master_Receive(&hi2c1, BH1750_ADRESSE, rx, 2, 10) != HAL_OK)
  {
    BH1750_Init(); /* relance si le capteur vient d'être branché */
    return 0;
  }
  *lux = (uint16_t)((((uint32_t)rx[0] << 8) | rx[1]) * 10 / 12); /* lux = brut / 1,2 */
  return 1;
}
```

Ce qui a changé et pourquoi

- **Calcul en entiers** : `× 10 / 12` au lieu de `/ 1,2` . Le résultat est le même au lux près, sans calcul à virgule flottante ni bibliothèque mathématique.
- Le délai I2C est de **10 ms**, et non plus `cfg.timeout` (30 s) : un capteur absent ne bloque plus la boucle. C'est `Capteurs_Tache()` qui gère le délai global de la mesure.
- Les deux octets sont assemblés par décalage (`<< 8`), plus rapide qu'une multiplication par 256.

7. Réception GPS et réglage de l'heure

Livrable 02

```
// Appelée par USART3_IRQHandler à chaque caractère reçu
FONCTION Capteurs_GPS_IRQ() : booléen
    TANT QUE Caractere_Disponible(USART3) FAIRE … (trame dans gps_trame)
FONCTION GPS_Lire_Position() : booléen
    SI trame n'est pas "$GPGGA" ni "$GNGGA" ALORS RETOURNER FAUX
    SI champs[6] = "0" ALORS RETOURNER FAUX      // pas de fix
```

Programme C : capteurs.c

```c
/* Appelée par USART1_IRQHandler. Renvoie 1 si l'interruption a été traitée ici. */
uint8_t Capteurs_GPS_IRQ(void)
{
  USART_TypeDef *u = huart3.Instance;

  if (!gps_actif) return 0;

  if (u->ISR & (USART_ISR_ORE | USART_ISR_FE | USART_ISR_NE))
  {
    u->ICR = USART_ICR_ORECF | USART_ICR_FECF | USART_ICR_NECF; /* caractères perdus : on continue
*/
  }
  while (u->ISR & USART_ISR_RXNE)
  {
    char c = (char)(u->RDR & 0xFF);
    if (c == '\n' || c == '\r')
    {
      if (gps_index > 0)
      {
        gps_rx_buffer[gps_index] = '\0';
        if (!gps_data_ready) /* la trame précédente a été lue */
```

```
        {
          strcpy(gps_trame, gps_rx_buffer);
          gps_data_ready = 1;
        }
        if (!gps_rmc_ready && strncmp(gps_rx_buffer + 3, "RMC", 3) == 0)
        {
          strcpy(gps_rmc, gps_rx_buffer); /* lue par Capteurs_GPS_Horloge(), même hors mesure */
          gps_rmc_ready = 1;
        }
        gps_index = 0;
      }
    }
    else if (gps_index < GPS_BUFFER_SIZE - 1)
    {
      gps_rx_buffer[gps_index++] = c;
    }
    else
    {
      gps_index = 0; /* trame trop longue : on recommence */
    }
  }
  return 1;
}

/* Somme de contrôle NMEA (XOR entre '$' et '*') */
static uint8_t GPS_Checksum_OK(const char *t)
{
  uint8_t x = 0;
  const char *p = t + 1;
  if (t[0] != '$') return 0;
  while (*p && *p != '*') x ^= (uint8_t)*p++;
  return *p == '*' && strtoul(p + 1, NULL, 16) == x;
}
```

Ce qui a changé et pourquoi

- **USART3 → USART1** (D2/D8) : USART3 (PC10/PC11) n'est disponible que sur le connecteur Morpho, et D0/D1 sont réservées au ST-Link. D2/D8 sont sur le connecteur Arduino, faciles à câbler. Dans le code, le handle garde le nom `huart3` généré par CubeMX, mais son instance est `USART1` (`GPS_USART` dans main.h).
- L'interruption lit **directement les registres** `ISR` et `RDR` au lieu de passer par `HAL_UART_Receive_IT` : le traitement d'un caractère est très court, ce qui compte à 960 caractères par seconde.
- **Somme de contrôle XOR** des trames NMEA (`GPS_Checksum_OK()`) : une trame abîmée (caractère perdu) est rejetée au lieu de donner une fausse position.
- La dernière trame **RMC** est aussi gardée : `GPS_Regler_RTC()` en tire l'heure UTC, ajoute le fuseau (UTC+1), gère le passage au jour suivant et calcule le jour de la semaine (méthode de Sakamoto). L'heure n'est réglée que si elle ne l'a pas été à la main.

8. Enregistrement sur la carte SD

Livrable 02

```
FONCTION Logger_Ecrire(ligne, a, m, j) : SdEtat
    etat ← Logger_Monter()
    f_open(fichier, nom0, OUVRIR_OU_CREER)
    SI Taille + Longueur(ligne) > cfg.file_max_size ALORS Archiver(…)
    Aller_A_La_Fin(fichier) ; f_write(fichier, ligne) ; f_close(fichier)
```

Programme C : logger.c

```
SdEtat Logger_Monter(void)
{
  if (monte) return SD_OK;
  if (f_mount(&fs, "0:/", 1) != FR_OK)
  {
    f_mount(NULL, "0:/", 0);
    return SD_ERREUR;
  }
  monte = 1;
  return SD_OK;
}

SdEtat Logger_Ecrire(const char *ligne, uint8_t annee, uint8_t mois, uint8_t jour)
{
  char nom0[24];
  FIL fil;
  UINT ecrit;
  FRESULT fr;
  UINT len = (UINT)strlen(ligne);
  SdEtat etat;

  if ((etat = Logger_Monter()) != SD_OK) return etat;

  snprintf(nom0, sizeof(nom0), "%02u%02u%02u_0.LOG", annee, mois, jour);

  fr = f_open(&fil, nom0, FA_OPEN_ALWAYS | FA_WRITE);
  if (fr != FR_OK) return Erreur(fr);

  if (f_size(&fil) > 0 && f_size(&fil) + len > (FSIZE_t)cfg.file_max_size)
  {
    f_close(&fil);
    if ((etat = Archiver(nom0, annee, mois, jour)) != SD_OK) return etat;
    fr = f_open(&fil, nom0, FA_CREATE_ALWAYS | FA_WRITE);
    if (fr != FR_OK) return Erreur(fr);
  }

  fr = f_lseek(&fil, f_size(&fil));
  if (fr == FR_OK) fr = f_write(&fil, ligne, len, &ecrit);
  if (fr != FR_OK)
  {
    f_close(&fil);
    return Erreur(fr);
  }
  fr = f_close(&fil);
  if (fr != FR_OK) return Erreur(fr);
  if (ecrit < len) return SD_PLEINE; /* volume plein */
  return SD_OK;
}
```

Ce qui a changé et pourquoi

- `f_mount(&fs, "0:/", 1)` : le **1 force le montage immédiat**. Une carte absente est détectée tout de suite, au démarrage, et pas seulement à la première écriture.
- Chaque appel FatFs est **vérifié** ; en cas d'erreur, la carte est démontée puis remontée à l'écriture suivante (`Erreur()`). Une carte retirée puis remise repart sans RESET.
- Le nombre d'octets réellement écrits (`ecrit < len`) est contrôlé : c'est ainsi qu'une carte pleine est reconnue.
- Le fichier est **fermé après chaque ligne** : une coupure de courant ne peut abîmer que la ligne en cours.

9. Paramètres et console

Livrable 02

```
POUR i DE 0 À NB_PARAMS - 1 FAIRE
    SI nom = PARAMS[i].nom ALORS
        v ← EnEntier(arg)
        SI v < PARAMS[i].min OU v > PARAMS[i].max ALORS Afficher("Valeur invalide")
        SINON PARAMS[i].valeur ← v ; Config_Sauver()
```

Programme C : config.c (table des paramètres)

```
static const ParamDef PARAMS[] =
{
  {"LOG_INTERVAL",  &cfg.log_interval,  1,    1440, 10},
  {"FILE_MAX_SIZE", &cfg.file_max_size, 256,  1048576, 2048},
  {"TIMEOUT",       &cfg.timeout,       1,    300,  30},
  {"LUMIN",         &cfg.lumin,         0,    1,    1},
  {"LUMIN_LOW",     &cfg.lumin_low,     0,    1023, 255},
  {"LUMIN_HIGH",    &cfg.lumin_high,    0,    1023, 768},
  {"TEMP_AIR",      &cfg.temp_air,      0,    1,    1},
  {"MIN_TEMP_AIR",  &cfg.min_temp_air,  -40,  85,   -10},
  {"MAX_TEMP_AIR",  &cfg.max_temp_air,  -40,  85,   60},
  {"HYGR",          &cfg.hygr,          0,    1,    1},
  {"HYGR_MINT",     &cfg.hygr_mint,     -40,  85,   0},
  {"HYGR_MAXT",     &cfg.hygr_maxt,     -40,  85,   50},
};
```

Ce qui a changé et pourquoi

- Chaque ligne de la table contient un **pointeur** vers le champ de `cfg` : une seule boucle sert à afficher, modifier, valider et remettre à zéro les 12 paramètres. Ajouter un paramètre = ajouter une ligne.
- La table est `const` : elle est rangée en **flash** et n'occupe pas de RAM.
- La valeur est lue avec `strtol` et refusée si elle contient autre chose qu'un nombre (`LOG_INTERVAL=10` est rejeté). Les espaces autour de `=` sont acceptés, et `CLOCK=HH:MM` aussi.
- **Flash au format v2** : mot magique `0x57575702` + somme de contrôle par rotation/XOR. Une sauvegarde de l'ancien format (avec la pression) est relue et convertie automatiquement, donc les réglages ne sont pas perdus.

10. LED d'état

Livrable 02

```
// PWM logiciel : période 10 ms, 10 niveaux d'intensité
pas ← ms MOD 10
Allumer(pas < r, pas < g, pas < b)   // anode commune : allumé = broche à 0
```

Programme C : led.c

```
/* Allume (1) ou éteint (0) chaque couleur en tenant compte du type de LED */
static void Ecrire(uint8_t r, uint8_t g, uint8_t b)
{
  uint32_t allumees = (r ? LED_R_Pin : 0) | (g ? LED_G_Pin : 0) | (b ? LED_B_Pin : 0);
  uint32_t eteintes = (LED_R_Pin | LED_G_Pin | LED_B_Pin) & ~allumees;

#if LED_ANODE_COMMUNE
  /* anode commune : couleur allumée = broche à 0 */
  LED_R_GPIO_Port->BSRR = (allumees << 16) | eteintes;
#else
  /* cathode commune : couleur allumée = broche à 1 */
  LED_R_GPIO_Port->BSRR = allumees | (eteintes << 16);
```

```
#endif
}
```

Ce qui a changé et pourquoi

- `LED_Tick()` s'exécute **1000 fois par seconde** dans une interruption : elle doit être très courte. Les trois couleurs sont mises à jour par **une seule écriture** dans le registre `BSRR`, au lieu de trois appels à `HAL_GPIO_WritePin`. Les trois broches changent au même instant, sans scintillement.
- Le type de LED (anode ou cathode commune) est choisi par `#if LED_ANODE_COMMUNE` à la **compilation** : aucun test à l'exécution.

Récapitulatif des optimisations

Optimisation	Où	Gain
Veille `__WFI()` entre deux mesures	led_modes.c	Le processeur dort au lieu de tourner à vide : moins de consommation
Acquisition non bloquante	capteurs.c	Boutons et LED réactifs pendant une mesure
Délais courts (I2C 10 ms, DHT11 1 ms par front)	capteurs.c	Un capteur absent ne bloque jamais la station
Calculs en entiers	capteurs.c, led_modes.c	Pas de virgule flottante, calculs exacts et rapides
Accès direct aux registres (RDR, ISR, BSRR)	capteurs.c, led.c	Interruptions plus courtes
Sommes de contrôle (DHT11, NMEA, flash)	capteurs.c, config.c	Les données abîmées sont rejetées
Table `const` de paramètres	config.c	Code plus court, table en flash et non en RAM
Types de taille exacte (`uint8_t`, `int16_t`, `uint32_t`), variables `static`, `volatile` pour les interruptions	tous les modules	RAM économisée, variables privées à leur module, partage sûr avec les interruptions
Choix à la compilation (`#if`, `#define`)	main.c, led.c, config.h	Tests et options sans coût à l'exécution
Pression retirée	capteurs, config	Ligne plus courte, moins de code et de paramètres

Limite qui reste : le signal de départ du DHT11 (18 ms) utilise encore `HAL_Delay`, et la lecture des 40 bits se fait interruptions coupées (environ 5 ms par mesure). C'est négligeable devant une période de plusieurs minutes (voir chapitre 10).

7. Données, fichiers et paramètres

7.1 Ligne de mesures

Chaque mesure produit une seule ligne terminée par `\r\n`, champs séparés par un point-virgule (ouverture directe dans un tableur) :

```
JJ/MM/AAAA HH:MM:SS;TEMPERATURE;HUMIDITE;LUMINOSITE;POSITION
23/10/2026 15:04:13;25;15;11294;3643.20860N,00303.12064E
23/10/2026 09:16:24;22;17;6308;NA
```

Champ	Unité	Règle
Horodatage	heure locale	Lu dans la RTC au début de la mesure
Température	°C, entier	`NA` si le DHT11 ne répond pas ou si `TEMP_AIR=0`
Humidité	% HR, entier	Écrite si la température est entre `HYGR_MINT` et `HYGR_MAXT` et si `HYGR=1`
Luminosité	lux, entier	`NA` si le BH1750 ne répond pas ou si `LUMIN=0`
Position	NMEA	`3643.20860N` = 36° 43,2086' Nord. `NA` sans fix ou si le GPS n'est pas lu

7.2 Fichiers sur la carte SD

- Nom au format 8.3 : `AAMMJJ_n.LOG`, par exemple `261023_0.LOG` pour le 23 octobre 2026.
- On écrit toujours dans la révision 0 du jour ; rotation vers `_1` … `_9` à `FILE_MAX_SIZE`.
- Une date non réglée donne `000101_0.LOG` (01/01/2000). Si les 9 révisions du jour existent, la carte est signalée pleine pour ce jour.

7.3 Paramètres

Paramètre	Défaut	Bornes	Rôle
`LOG_INTERVAL`	10	1 … 1440	Minutes entre deux mesures
`FILE_MAX_SIZE`	2048	256 … 1 048 576	Taille maximale d'un fichier, en octets
`TIMEOUT`	30	1 … 300	Durée maximale d'une acquisition, en secondes
`TEMP_AIR`	1	0 … 1	Active la mesure de température
`MIN_TEMP_AIR` / `MAX_TEMP_AIR`	−10 / 60	−40 … 85	Plage de température cohérente (°C)
`HYGR`	1	0 … 1	Active la mesure d'hygrométrie
`HYGR_MINT` / `HYGR_MAXT`	0 / 50	−40 … 85	Plage de température où l'hygrométrie est enregistrée
`LUMIN`	1	0 … 1	Active la mesure de luminosité

`LUMIN_LOW` / `LUMIN_HIGH`	255 / 768	0 … 1023	Seuils de luminosité du cahier des charges

7.4 Sauvegarde en flash

La STM32L476 n'a pas d'EEPROM. Les paramètres sont écrits dans la dernière page de 2 Ko de la flash (banque 2, page 255, adresse `0x080FF800`), avec un mot magique (version du format) et une somme de contrôle. Au démarrage, une page vide ou abîmée entraîne le chargement des valeurs par défaut. La page n'est réécrite qu'à la modification d'un paramètre, ce qui ménage la flash.

8. Gestion des erreurs

Les erreurs sont détectées à chaque mesure et affichées sur la LED par un clignotement à 1 Hz : rouge, puis une seconde couleur. « Durées égales » = 0,5 s / 0,5 s ; « 2 fois plus long » = 0,33 s de rouge puis 0,67 s de l'autre couleur.

Prio.	Erreur	Condition de détection	Code LED
1	Horloge RTC	Lecture de l'heure ou de la date impossible	 rouge / bleu, égal
2	Accès carte SD	Carte absente, montage, ouverture ou écriture impossible	 rouge / blanc, blanc ×2
3	Carte SD pleine	Plus de place, ou 9 archives pour le jour	 rouge / blanc, égal
4	Capteur	DHT11 ou BH1750 sans réponse pendant 2 acquisitions consécutives	 rouge / vert, égal
5	GPS	Pas de position pendant 2 acquisitions consécutives où le GPS est lu	 rouge / jaune, égal
6	Données incohérentes	Température hors `MIN_TEMP_AIR` … `MAX_TEMP_AIR`, ou humidité hors 0…100 %	 rouge / vert, vert ×2

8.1 Principes retenus

- **Une seule erreur affichée** : la plus grave (`Erreurs_LED()` teste dans l'ordre des priorités).
- **Retour automatique** : la LED reprend la couleur du mode dès qu'une mesure est correcte, sans acquittement.
- **Deux échecs avant l'alerte** pour les capteurs et le GPS : une perte ponctuelle ne déclenche pas de fausse alarme.
- **Pas de blocage** : la station continue de mesurer et d'enregistrer ce qui est disponible (`NA` pour le reste).
- **Contrôle des données reçues** : somme de contrôle du DHT11, somme XOR des trames NMEA, erreurs UART (débordement, trame) effacées sans arrêter la réception.

9. Performances

Valeurs issues des constantes du programme, des fiches techniques et des essais sur la maquette. La consommation électrique n'a pas été mesurée.

9.1 Cadences et temps de réponse

Grandeur	Valeur	Remarque
Période en mode standard	1 à 1440 min (10)	`LOG_INTERVAL`
Période en mode économique	2 × `LOG_INTERVAL`	GPS lu une mesure sur deux
Période en mode maintenance	5 s	plus la durée de l'acquisition
Durée d'une acquisition (tout OK, GPS avec fix)	environ 1 à 2 s	le GPS envoie une position par seconde
Durée maximale d'une acquisition	`TIMEOUT` (30 s)	GPS sans fix ou capteur absent
Changement de mode	5 s d'appui	anti-rebond 30 ms
Premier fix GPS (à froid, extérieur)	1 min 30 mesuré	7 satellites vus, 4 utilisés

9.2 Stockage

Grandeur	Valeur (paramètres par défaut)
Taille d'une ligne	56 octets (34 avec position `NA`)
Lignes par jour en mode standard	144 lignes, environ 8 Ko
Lignes par fichier (`FILE_MAX_SIZE` = 2048)	36 lignes, soit 6 h de mesures
Volume maximal par jour	10 fichiers × `FILE_MAX_SIZE` = 20 Ko

> **Règle de réglage**
> Une journée est limitée à 10 fichiers : il faut `LOG_INTERVAL` ≥ 4 min avec `FILE_MAX_SIZE` = 2048. Pour une mesure par minute, régler `FILE_MAX_SIZE` à 16384.

9.3 Énergie

- Entre deux mesures, le processeur est mis en veille (`__WFI()`) et ne se réveille qu'aux interruptions : SysTick, réception GPS, bouton B1.
- Le mode économique divise par deux le nombre de mesures et par quatre le nombre de lectures GPS.
- Le poste principal reste le GPS, alimenté en permanence pour garder ses satellites.

Ressources utilisées : horloge 80 MHz (HSI 16 MHz + PLL) ; I2C1, SPI1, USART1, USART2, RTC, FLASH, 8 GPIO, compteur DWT ; tampons 3 × 128 octets (GPS), 96 octets (ligne), 64 octets (console) ; 1 page de flash de 2 Ko pour les paramètres.

10. Limites et évolutions

10.1 Limites du prototype

Limite	Conséquence et solution envisagée
Pas de pile sur VBAT	L'heure est perdue à la coupure. Reprise par le GPS ; une pile CR2032 la conserverait même sans satellite.
Plage du DHT11 (0 à 50 °C)	Insuffisante en mer par grand froid. Un capteur I2C SHT31 ou BME280 (qui ajoute la pression) serait plus adapté.
Pression non mesurée	Grandeur du cahier des charges non disponible avec le matériel fourni.
Veille légère	`__WFI()` réveille le processeur chaque milliseconde (SysTick). Le mode STOP réveillé par la RTC consommerait moins, au prix d'une LED pilotée par timer matériel.
DHT11 lu interruptions masquées	Quelques caractères GPS peuvent être perdus pendant 5 ms ; la trame est rejetée par la somme de contrôle et la suivante est utilisée.
9 archives par jour	Au-delà, les mesures du jour ne sont plus enregistrées : respecter la règle de réglage (9.2).

10.2 Intégration des modules complémentaires

Le sujet annonce quatre modules : température de l'eau (analogique), courant marin (I2C), vent (I2C) et particules fines (2 fils). L'architecture permet de les ajouter sans toucher à la logique des modes :

1. Ajouter un champ (valeur + indicateur de validité) dans la structure `Mesures`.
2. Ajouter une étape de lecture non bloquante dans `Capteurs_Tache()`, avec son compteur de timeouts.
3. Ajouter une colonne dans `Capteurs_Formater()` (la ligne reste unique).
4. Ajouter les paramètres d'activation et de seuils dans la table de `config.c` : ils deviennent réglables par la console et sauvegardés en flash.

Les modules I2C se brancheront sur le même bus que le BH1750 (D14/D15), à condition d'avoir une adresse différente de 0x23.